Kiln Research Guide: Evaluating REA (Reverse Engineer Anything)

Researched 2026-10-12.

Overview: Separating Demonstration Claims from Verified Reality

The viral video by Nick Saraev on YouTube (https://youtu.be/dAzs_rU6080) popularized REA (*Reverse Engineer Anything*) by pitching it as an end-of-software paradigm shift, showing a side-by-side rebuild of macOS Preview using Claude. However, cross-referencing these claims against the official morluto/rea GitHub Repository (https://github.com/morluto/rea) clarifies that REA is a local orchestration interface exposing battle-tested binary, JavaScript, and managed analysis frameworks to Model Context Protocol (MCP) clients.

---

1. What the Reel Claims

- **Agent Autonomy & Speed:** Claims REA allows any connected agent to autonomously reverse-engineer complex interfaces (such as macOS Preview or an Apple TV remote) in under 45 minutes.
- **Universal Decompilation:** Asserts that REA makes most software 'effectively open source' by combining Ghidra, Hopper, and IDA into an iterative rebuilding loop.
- **Control Run Discrepancy:** The reel ran an informal head-to-head race between Claude with REA vs. vanilla Claude/Opus rebuilding macOS Preview. While the non-REA run produced code first, the reel claimed the REA version achieved higher interactive fidelity (smoother drawing, exact layout).
- **Hype vs. Substance:** The creator noted that star velocity graphs dominate the conversation, citing community claims (like a total Photoshop rebuild or games) that may conflate standard LLM UI prototyping with actual decompilation evidence.

2. What Official Documentation Confirms

- **Architecture:** According to the REA documentation (https://github.com/morluto/rea), REA acts as an MCP server and CLI utility (`rea-agents`) that routes structured requests to local tools and returns structured evidence (assembly, pseudocode, strings, and module boundaries) with explicit unknowns.
- **Local Execution:** REA runs locally under user process permissions. Static JavaScript and .NET analysis require no external engines, but deep native analysis explicitly depends on existing local installs of Ghidra, Hopper Disassembler, or IDA Pro.
- **Tooling Lineage:** The underlying engines are long-standing tools (IDA originating in 1991, Hopper circa 2011, and NSA's Ghidra open-sourced in 2019). REA unifies their outputs via standard interfaces.
- **Verifiable Showcases:** The official repository highlights granular algorithmic recovery rather than full turnkey clones: extracting a sound panning formula from *DX-Ball* (3,205 test cases matching 63 compiled bytes) and tracing Notion's Electron IPC clipboard bridge.

3. Uncertainties and Community Discrepancies

- **Reel Benchmark Validity ($n=1$):** The reel's macOS Preview comparison was conducted concurrently on a single machine without isolated environments, blinded scoring, or resource

profiling. The creator even noted the vanilla Claude run may have suffered lag due to Ghidra's CPU consumption in the background.

- **Misattributed Viral Projects:** Commenters noted that viral projects attributed to REA (such as the *Artcraft* demo) were built without REA, relying purely on visual prompting and written specifications.
- **Star Velocity vs. Stability:** While the repo has exceeded 110,000 GitHub stars, rapid release cycles require frequent updates (`rea update`), and native provider setups vary significantly across macOS, Linux, and Windows.

4. Technical Prerequisites & Operating Limits

- **Runtime Requirements:** Node.js 22.x (≥22.19), 24.x (≥24.11), or 26+, alongside npm.
- **Native Dependencies:** Requires user-supplied Ghidra, Hopper, or IDA Pro for compiled Mach-O/ELF/PE binaries. Android targets require headless JADX and a JDK; WASM targets require WABT 1.0.42.
- **Security Warning:** '*This is not a sandbox.*' Analyzing untrusted binaries or running runtime captures executes with the current host user permissions.
- **Legal & License Guardrails:** REA's MIT license does not override third-party software End User License Agreements (EULAs), copyright protections, or anti-circumvention provisions. Analysis must be restricted to owned code, open-source software, or authorized research targets.

---

5. Verified Command Reference

Only the following commands are confirmed by official documentation from the REA project repository (https://github.com/morluto/rea):

Interactive Agent Setup (Registers MCP & installs workflow instructions):

```
npx rea-agents setup
```

Global Installation:

```
npm install --global rea-agents
rea --help
```

Update Global CLI:

```
rea update
```

Update Agent MCP Instructions via npx:

```
npx rea-agents@latest setup
```

Static JavaScript / Electron Directory Inspection (Zero native engine requirement):

```
npx -y rea-agents@latest analyze-javascript-application /absolute/path/to/app --json
```

(Note: Natural language prompts like 'tell your agent to download REA' from the reel are unverified informal descriptions, not documented CLI commands).

---

6. Common Pitfalls & Mistakes

1. **Expecting Whole-Application Code Recovery:** REA provides pseudocode fragments, references, and AST structures; it does not reconstruct original, fully maintainable source trees in a single pass.
2. **Ignoring Provider Timeouts:** Running Ghidra on large native binaries often times out unless configured with environment overrides such as `REA_GHIDRA_STARTUP_TIMEOUT_MS`.
3. **Licensing Assumptions:** Assuming REA bundles commercial tools like IDA Pro or Hopper. Users must supply their own authorized licenses.
4. **Hallucination vs. Evidence:** Failing to demand citation of decompiled routines from the agent, leading to synthetic approximations rather than true functional reverse engineering.

---

7. Reusable Experimental Prompt

```
You are an auditing assistant evaluating binary/application features using REA.
Target: [Path to Authorized App or Open-Source Component]
Task:
1. Use REA tools to trace the implementation of [Specific Feature/Function Name].
2. Do not write replacement code until you produce an Evidence Report detailing:
   - Decompiled pseudocode or disassembly blocks discovered.
   - Identified modules, exported symbols, or IPC routes.
   - Documented limitations or unknown behaviors.
3. Based exclusively on the extracted logic, produce an isolated, minimal clean-room
implementation of that specific function with unit tests validating parity against original
inputs.
```

Original sources

- https://youtu.be/dAzs_rU6080